

Drone de surface à voile

Rapport technique — Développement du firmware et de l'électronique

Plateforme :	LilyGO T-Beam V1.1 (ESP32 + SX1276 + GPS u-blox + AXP192)
Langage :	Arduino / C++, cible ESP32 Arduino Core 3.x
Membres du groupe :	Facundo Arito
Date :	June 13, 2026

Résumé

Ce document retrace l'intégralité du développement informatique et électronique du drone de surface à voile basé sur le module T-Beam d'Espressif/LilyGO. Il couvre le concept physique du véhicule, le code original et ses limitations, la réécriture complète de l'architecture logicielle, les deux révisions du PCB, le mode manuel unifié avec propulsion bidirectionnelle, la navigation autonome à voile, l'interface de station sol (IHM) et, de manière détaillée, **tous les problèmes** rencontrés au cours du projet ainsi que les solutions mises en place. Une section finale réunit les problèmes encore ouverts et les options d'amélioration proposées pour les résoudre.

Projet de recherche (PER) — Drone autonome d'acquisition de données marines

Table des matières

1	Guide d'utilisation pour le prochain groupe de travail	2
1.1	Matériel nécessaire	2
1.2	Mise en marche du drone (bateau)	2
1.3	Programmation (flash) des T-Beam	3
1.4	Connexion des deux T-Beam	4
1.5	Utilisation de l'IHM sous Linux	4
1.5.1	Programmes à installer (Linux)	4
1.6	Utilisation de l'IHM sous Windows	6
1.6.1	Programmes à installer (Windows)	6
2	Présentation du projet	7
2.1	Concept physique	7
2.2	Objectif du projet	8
2.3	Matériel principal	8
3	Code original — <code>boat.ino</code>	8
3.1	Architecture et fonctionnement	8
3.2	Problèmes et limitations identifiés	9
3.2.1	Bibliothèque ESP32Servo — conflit LEDC / interruptions . . .	9
3.2.2	Affectation des broches RC — conflit I2C / AXP192	9
3.2.3	Broche batterie GPI035 — drain de 20 mA continu	9
3.2.4	Broche LoRa RST — erreur de documentation	9
3.2.5	Gestion de l'ESC — sans armement ni limitation de variation .	10
4	PCB Version 1 — Carte d'interface originale	10
5	Réécriture du firmware — Architecture Post-Claude	10
5.1	Motivations	10
5.2	Architecture en couches	11
5.3	Décisions techniques fondamentales	12
5.3.1	MCPWM obligatoire — jamais LEDC	12
5.3.2	Interruptions RC avec protection portMUX	12
5.3.3	Toutes les consignes d'actionneurs en microsecondes	13
5.3.4	Slew rate — protection contre les à-coups	13
6	Phase 1 — Contrôle manuel par radio	13
6.1	Réalisation	13

6.2	Modes de vol (sélecteur CH5) — version unifiée actuelle	13
6.3	Mode manuel unifié	14
6.4	Propulseur bidirectionnel (marche avant / arrière)	15
6.5	Résultats des tests — Phase 1	15
7	Phase 2 — GPS et navigation	16
7.1	Problème critique — GPS muet après configuration en flash	16
7.2	Problème critique — antenne active non alimentée	16
7.3	Note sur le démarrage à froid	17
8	Phase 3 — LoRa et télémétrie	17
8.1	Protocole JSON — heartbeat actuel	17
8.2	Commandes (sol → drone)	17
9	PCB Version 2 — Améliorations matérielles	18
9.1	Correspondance des broches (PCB V2)	18
9.2	Amplification de niveau logique — transistors BC548	19
9.3	Mesure de batterie — diviseur haute impédance	19
9.4	AXP192 — I2C persistant	19
10	Navigation autonome (Phases 5 et 6)	20
10.1	Phase 5 — Estimation du vent à partir du track GPS	20
10.2	Phase 6 — Algorithme de navigation à voile	20
11	Interface de station sol (IHM)	21
12	Problèmes résolus et solutions détaillées	21
12.1	Alimentation du T-Beam — impossible d'alimenter par les broches . . .	21
12.2	Moteur qui fonctionne “par impulsions” — coupure par basse tension .	22
12.3	Échec de programmation du bateau — brownout pendant le flash . . .	23
12.4	Autres problèmes de firmware déjà résolus	23
12.4.1	DUTY_MODE_1 sur OPR_B — servo du safran inopérant . .	23
12.4.2	Seuil d'armement de l'ESC trop bas	23
12.4.3	Mode bloqué en Automatic — confusion CH4/CH5	23
12.4.4	Port série occupé lors de la programmation	23
13	Problèmes rencontrés — options d'amélioration	24
13.1	Portée LoRa très faible à l'intérieur du boîtier électronique	24
13.2	Incohérences du GPS	24
13.3	Sans mesure du vent ni de la position de la voile	25
13.4	Peu d'espace pour les batteries dans le boîtier	25

14 État actuel du système	26
14.1 Valeurs de calibration actuelles (résumé)	26

1. Guide d'utilisation pour le prochain groupe de travail

Cette section est un **guide pratique de mise en marche** destiné au groupe suivant qui reprendra le projet. Elle explique quel matériel est nécessaire, comment allumer et piloter le drone, comment programmer et connecter les deux cartes T-Beam, et comment lancer l'interface de station sol (IHM) aussi bien sous Linux que sous Windows. Il est recommandé de la lire **en entier** avant le premier allumage.

1.1. Matériel nécessaire

Élément	Détail
Drone (“bateau”)	T-Beam montée dans la coque sur le PCB V2. Convertisseur USB-série CP2104, numéro de série 02126CF1
Transcepteur (station sol)	Deuxième T-Beam avec le sketch <code>transceiver.ino</code> . CP2104 série 01C00B54
Radio RC	Émetteur Pro-Tronik PTR-6A + récepteur R8X (2,4 GHz)
Batterie	LiPo 2S chargée ($\approx 8,4$ V) + porte-batteries de la coque. Indispensable pour un fonctionnement stable
Ordinateur	Linux (recommandé) ou Windows, avec Docker, Python 3 et navigateur
Câbles USB	Un de bonne qualité pour le transcepteur ; un autre pour programmer / alimenter le bateau

1.2. Mise en marche du drone (bateau)

1. **Charger la LiPo** complètement et la connecter au **bornier du PCB** du bateau (le GND est indiqué sur le PCB).
2. **Allumer d'abord l'émetteur RC** et vérifier que le sélecteur de mode (CH5) est en **Manuel** (position basse). Le **premier essai doit toujours être manuel** afin de tester tous les actionneurs.
3. **Mettre le bateau sous tension.** Connecter l'USB de la T-Beam (soit à

l'alimentation du PCB, soit à un PC pour lire le port série). Au démarrage, il imprime sur le port série des lignes d'état (p. ex. [LORA] OK).

4. **Attendre le fix GPS** : la LED TIMEPULSE du NEO-6M clignote à 1 Hz quand il y a un fix. Avec la LiPo connectée, le fix arrive en quelques secondes (démarrage à chaud) ; sans LiPo, chaque allumage est un démarrage à froid d' ≈ 2 min.

5. Piloter en manuel :

- **CH5 (mode)** : haut = Automatique, milieu = Sail (inerte, tout au neutre), bas = Manuel.
- **CH2 (voile)** : bascule la voile entre bâbord et tribord ($\pm 10^\circ$).
- **CH4 (safran)** : positionne le treuil du safran ($\pm 90^\circ$).
- **CH3 (moteur, bidirectionnel)** : centre = arrêt, une extrémité = +100 % avant, l'autre = -100 % arrière. Nécessite l'ESC en mode bidirectionnel (voir point suivant).

Sécurité : en cas de perte du signal RC, le drone passe en *Failsafe*, qui **navigue de façon autonome**. Comme le mode autonome n'a jamais été validé sur l'eau, le premier essai doit se faire en manuel, en eaux calmes et avec un plan de récupération (p. ex. barque ou cordage).

1.3. Programmation (flash) des T-Beam

Les deux cartes utilisent le même noyau (FQBN = `esp32:esp32:t-beam`). Le bateau porte le sketch `main/` ; le transcepteur porte `transceiver/`.

Listing 1: Compiler et flasher avec arduino-cli

```
# (chemins relatifs au dossier racine du projet = Post-Claude)
# Compiler le firmware du bateau
~/bin/arduino-cli compile --fqbn esp32:esp32:t-beam main

# Flasher le bateau (ajuster le port)
~/bin/arduino-cli upload --port /dev/ttyUSB0 --fqbn esp32:esp32:t-beam main

# Idem pour le transcepteur
~/bin/arduino-cli upload --port /dev/ttyUSB0 --fqbn esp32:esp32:t-beam transceiver
```

- **Identifier chaque carte par son numéro de série** (sous Linux : `ls -l /dev/serial/by-id/`). Bateau = 02126CF1, transcepteur = 01C00B54.
- **Si le flash échoue** (“*chip stopped responding*”, le port se ré-énumère) : c’est un *brownout* par alimentation insuffisante. Entrer en mode bootloader manuel (maintenir **BOOT/IO0** en appuyant sur **RST** ou en reconnectant l’USB). Voir Section 12.1.
- Ne pas avoir de moniteur série ouvert sur le port pendant la programmation.

1.4. Connexion des deux T-Beam

- Le **transcepteur** se connecte en **USB à l’ordinateur** de sol. Il apparaît comme `/dev/ttyUSBx` sous Linux ou `COMx` sous Windows.
- Le **bateau ne se connecte pas par câble** pendant l’opération : il communique avec le transcepteur par **radio LoRa à 433 MHz**. Les deux firmwares utilisent déjà les mêmes paramètres radio (433 MHz, SF7, BW125 kHz).
- Le transcepteur retransmet à l’IHM chaque heartbeat du bateau (et lui injecte le RSSI mesuré au sol) et transmet au bateau les commandes de l’IHM.

Ne pas ouvrir un moniteur série (gtkterm, le moniteur de l’IDE Arduino, un script pyserial, etc.) sur le port du transcepteur **pendant que l’IHM tourne** : les deux lecteurs se disputent le port et corrompent le JSON de télémétrie.

1.5. Utilisation de l’IHM sous Linux

1.5.1. Programmes à installer (Linux)

Programme	Pour quoi / comment
Docker Engine + plugin docker compose	Lance la base de données MongoDB (image <code>mongo:7</code> , téléchargée automatiquement la première fois). Sous Ubuntu/Debian : <code>sudo apt install docker.io docker-compose-plugin</code> . Ajouter l’utilisateur au groupe : <code>sudo usermod -aG docker \$USER</code> (puis rouvrir la session) pour ne pas utiliser <code>sudo</code>
Python 3 + venv + pip	Exécute le serveur et le pont série. <code>sudo apt install python3 python3-venv python3-pip</code>

Programme	Pour quoi / comment
Paquets Python	S'installent dans un environnement virtuel à partir de <code>requirements.txt</code> : <code>fastapi</code> , <code>uvicorn</code> , <code>pymongo[asyncio]</code> , <code>pyserial</code> , <code>python-dotenv</code>
Navigateur web	Pour ouvrir l'interface (Firefox, Chrome, etc.)

Préparation (une seule fois) — créer l'environnement virtuel et installer les paquets Python (chemins relatifs à la racine **Post-Claude**) :

Listing 2: Installation initiale de l'IHM sous Linux

```
cd IHM
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Démarrage et arrêt — connecter le transcepteur en USB et, depuis le dossier **IHM/**, exécuter :

Listing 3: Demarrer et arreter l'IHM sous Linux

```
cd IHM
./start_ihm.sh                # autodetecte le transcepteur par son
    numero de serie
# ou forcer le port :
./start_ihm.sh /dev/ttyUSB0

# ... a la fin :
./stop_ihm.sh                # libere le port 5000 et arrete
    MongoDB (Docker)
```

- `start_ihm.sh` lance MongoDB (conteneur Docker via `docker compose`), `serial_link.py` (lit/écrit le port série) et `webserver.py` (serveur web), et ouvre le navigateur sur `http://localhost:5000`.
- Sur la carte : voir la télémétrie en direct, définir des waypoints au clic, envoyer la route et les commandes (naviguer, arrêter, mesurer le vent, etc.).
- Commutateur **Sim / Réel** : en “Réel” il utilise le transcepteur physique ; en “Sim” il lance un simulateur de bateau (sans matériel).
- Le port du transcepteur est **autodéecté** par son numéro de série (01C00B54), il n'est donc normalement pas nécessaire de l'indiquer.

1.6. Utilisation de l'IHM sous Windows

L'IHM a été développée sous Linux (les scripts de démarrage sont en **bash**), c'est pourquoi sous Windows le plus simple est de l'exécuter dans WSL2.

1.6.1. Programmes à installer (Windows)

Programme	Pour quoi / comment
WSL2 (Ubuntu)	Sous-système Linux dans Windows ; permet d'utiliser les mêmes scripts bash . Installer avec wsl -install dans PowerShell (en administrateur)
Docker Desktop	Lance MongoDB. L'installer et activer l' intégration avec WSL2 dans ses paramètres
usbipd-win	Partage le port USB du transcepteur avec WSL (Windows ne le passe pas seul). Installer avec winget install usbipd
Python 3 + paquets	Déjà présent dans l'Ubuntu de WSL ; installer les paquets comme sous Linux (requirements.txt). En Windows natif, installer Python depuis python.org
Navigateur web	Pour ouvrir http://localhost:5000

Voie recommandée — WSL2 + Docker Desktop : dans Ubuntu (WSL) on suit **exactement les mêmes étapes que sous Linux** (créer le **.venv**, installer **requirements.txt**, **./start_ihm.sh**). Pour que WSL “voie” le transcepteur, il faut attacher l'USB depuis PowerShell :

Listing 4: Attacher l'USB du transcepteur a WSL (PowerShell admin)

```
usbipd list                                # voir le BUSID du CP210x (
    transcepteur)
usbipd bind --busid <BUSID>
usbipd attach --wsl --busid <BUSID>      # dans WSL apparaît comme /dev/
    ttyUSBx
```

Alternative — Windows natif (sans WSL) : installer Docker Desktop et Python pour Windows et lancer les composants à la main : **docker compose up** pour MongoDB, puis **python serial_link.py** et **python webserver.py**. Le port série sera COMx (à vérifier dans le Gestionnaire de périphériques) et il faut ajuster **SERIAL_PORT_REAL** dans le fichier **.env** à ce COMx. Dans les deux cas, l'interface s'ouvre sur **http://localhost:5000**.

Réseau WiFi — EDUROAM ne fonctionne pas. Comme l'IHM utilise MongoDB (dans Docker) et la pile réseau locale, sur le réseau institutionnel **EDUROAM l'IHM ne fonctionne pas** : EDUROAM applique un isolement des clients et des restrictions réseau (pare-feu/proxy) qui empêchent le bon fonctionnement de Docker/MongoDB et de la communication locale nécessaire.

Solution (testée par le groupe) : connecter l'ordinateur qui exécute l'IHM à un réseau sans ces restrictions. En pratique, **partager les données mobiles du téléphone (hotspot / partage de connexion) vers l'ordinateur** : avec le hotspot du téléphone, l'IHM démarre et fonctionne correctement.

2. Présentation du projet

2.1. Concept physique

Le drone est un véhicule de surface propulsé principalement par une voile. Sa conception est originale : la voile est à **profil d'aile** et tourne librement autour d'un axe vertical solidaire de la coque. Son aérodynamique passive oriente automatiquement le profil de sorte que le vent arrive toujours à environ 45° de l'axe longitudinal du drone, sans besoin d'un contrôle actif de l'angle de la voile. La manœuvre se fait avec deux actionneurs principaux :

- **Servo aileron (voile) — Futaba S3003** : contrôle un petit volet sur le bord de fuite de la voile. Sa course utile est strictement de $\pm 10^\circ$. La déflexion détermine de quel côté (bâbord ou tribord) la voile pousse le bateau, choisissant de fait l'amure sur laquelle il navigue. **Ce n'est pas un contrôle continu** : seules les deux positions extrêmes ont un sens physique, donc on la modélise comme un état binaire (+1 / -1).
- **Servo safran / rotor — Graupner Regatta ECO II (mod. 5176)** : c'est l'actionneur principal de cap (lacet). C'est un *treuil de voile multi-tour positionnel* : le signal PWM commande une **position** de l'arbre (et non une vitesse). Centre mécanique = 1500 μs , course totale = 6 tours, et il maintient la position sous charge comme un servo standard.
- **Propulseur (ESC)** : utilisé uniquement en l'absence de vent suffisant. Permet la propulsion à hélice en secours. Sur le PCB V2 on utilise un seul ESC (le second a été supprimé).

Le système **ne mesure pas** la direction du vent directement : il la *déduit* à partir du track GPS et de l'état des actionneurs. L'estimation nécessite un déplacement minimal de 30 m. Ce manque de capteurs est central dans le projet et est repris en Section 13.

2.2. Objectif du projet

Le projet est un PER (Projet d'Étude et de Recherche) dont le but final est l'**acquisition autonome de données marines** : le drone doit atteindre des points de passage (*waypoints*) GPS en naviguant à la voile, collecter des données de capteurs et les transmettre par radio LoRa à une station sol.

2.3. Matériel principal

Composant	Description
LilyGO T-Beam V1.1	Module intégrant ESP32 WROVER, LoRa SX1276 433 MHz, GPS u-blox NEO-6M, AXP192 (gestion d'énergie) et porte-batteries 18650
Futaba S3003	Servo aileron de la voile, centre à 1520 μ s, $\pm 10^\circ$
Graupner Regatta ECO II	Treuil de safran multi-tour, 1500 μ s = centre, 1000–2000 μ s = plage, jusqu'à 3,3 A en charge
Pro-Tronik PTR-6A	Émetteur 6 voies FHSS 2,4 GHz, PWM 1000–2000 μ s
Pro-Tronik R8X	Récepteur 8 voies, PWM standard 50 Hz
Pro-Tronik Black Fet	ESC de la série, programmation par carte EPRG-3
Antenne GPS	Taoglas active, connecteur u.FL
PCB v1 / PCB v2	Carte d'interface sur mesure (KiCad 9)

3. Code original — `boat.ino`

3.1. Architecture et fonctionnement

Le code original (`boat.ino`, du premier groupe, hors de ce dépôt), écrit par Mohamed EL JILY, constitue le point de départ du projet. Il était organisé en modules :

Gps, ServoControl, RadioReceiver, MotorControl et la logique de navigation dans boat.ino. Il incluait déjà une logique de navigation à la voile (angle relatif vent/cap, changement d'amure automatique, lofer/abattre) et la réception de waypoints par LoRa en JSON.

3.2. Problèmes et limitations identifiés

À la reprise du projet, plusieurs problèmes bloquants ont été identifiés qui, conjointement avec le besoin de nouvelles fonctionnalités discutées avec le client et le professeur, ont motivé la réécriture complète du firmware.

3.2.1. Bibliothèque ESP32Servo — conflit LEDC / interruptions

La bibliothèque **ESP32Servo** utilise le périphérique **LEDC** de l'ESP32. LEDC est **incompatible** avec le décodage du PWM par interruptions GPIO utilisé pour lire la radio : lorsque LEDC est actif, les interruptions du récepteur RC sont perturbées et les canaux lus **reviennent constamment à zéro**, même si les signaux d'entrée sont corrects à l'oscilloscope. Cela rend impossible de contrôler les servos et de lire la radio en même temps. C'était le problème le plus critique du projet.

3.2.2. Affectation des broches RC — conflit I2C / AXP192

Le code original affectait des canaux RC à GPIO21 et GPIO22, qui sont les broches I2C internes du T-Beam utilisées par l'AXP192 (gestionnaire d'énergie). De plus, GPIO23 est relié en interne au RESET du SX1276 (LoRa) : attacher une interruption sur cette broche pouvait provoquer des resets espérés de la radio.

3.2.3. Broche batterie GPIO35 — drain de 20 mA continu

Le T-Beam V1.1 possède un diviseur résistif de *faible impédance* ($\approx 370 \Omega$ total) soudé en permanence sur GPIO35. Même sans lire l'ADC, ce diviseur consomme ≈ 20 mA en continu ($\approx 1,8$ W) directement de la batterie. **Ne jamais lire GPIO35.**

3.2.4. Broche LoRa RST — erreur de documentation

La documentation de LilyGO et de nombreux exemples indiquent GPIO14 pour le RESET du SX1276. Le hardware du T-Beam V1.1 câble physiquement le RESET sur GPIO23.

3.2.5. Gestion de l'ESC — sans armement ni limitation de variation

Le code original n'implémentait aucune fonction de l'ESC, car c'était une fonctionnalité prévue pour plus tard, tout comme la mesure de batterie.

4. PCB Version 1 — Carte d'interface originale

La première carte réalisait l'interface entre le T-Beam et les actionneurs (connecteurs 3 broches pour servos/ESC, connecteurs RC, régulateur abaisseur 5 V Pololu, bornes de puissance). Ses problèmes sont résumés ci-dessous.

Problème	Impact
PWM 3,3 V sans amplification de niveau	Servos et ESC (logique 5 V) peuvent ne pas détecter correctement les impulsions
Broches RC sur GPIO21/GPIO22	Conflit I2C/AXP192
ESC sur GPIO32/GPIO33	Coïncident avec LoRa DIO1/DIO2 du T-Beam
GPIO35 pour la batterie	Diviseur de $370\ \Omega = 20\ \text{mA}$ continus
Pas de protection de polarité	Risque en cas d'alimentation incorrecte

5. Réécriture du firmware — Architecture Post-Claude

5.1. Motivations

La réécriture complète vise à corriger toutes les limitations identifiées et à préparer l'architecture pour les phases autonomes. Objectifs :

1. Utiliser **MCPWM** au lieu de LEDC pour les sorties d'actionneurs.
2. Décodage RC **entièrement par interruptions** avec protection ISR.

3. Architecture orientée objet en couches, extensible par phases.
4. Armement de l'ESC et limitation de variation (*slew rate*).
5. Gestion propre de l'énergie via AXP192.
6. GPS, LoRa et batterie fonctionnant simultanément.

5.2. Architecture en couches

Listing 5: Arborescence du projet (racine = Post-Claude)

```

Post-Claude/                <- dossier racine du projet
  main/main.ino             <- Point d'entree (begin / update)
  main/src/
    app/DroneApp.cpp         <- Orchestrateur (scheduler logiciel)
    config/BoardConfig.h     <- Affectation des GPIO et seuils RC
    config/Calibration.h     <- Valeurs en microsecondes (servo/ESC
    )
    core/Types.h             <- Types partagés (RcFrame,
    ActuatorCommand)
    control/ModeManager      <- CH5 -> mode de controle
    control/ManualController <- Mode manuel unifié (voile+safran+
    moteur)
    control/AutoController   <- Enveloppe de la navigation autonome
    drivers/RcReceiver        <- Lecture PWM par interruptions
    drivers/McpwmActuators    <- Sorties MCPWM (voile, safran, ESC)
    drivers/AxpPower          <- Init AXP192 (rails GPS/LoRa/3.3V)
    drivers/BatteryAdc        <- ADC tension batterie
    drivers/GpsUart           <- Lecture GPS (TinyGPSPlus + UBX)
    drivers/LoRaRadio         <- Driver SX1276
    comm/LoRaComm             <- Protocole JSON LoRa (heartbeat +
    commandes)
    navigation/               <- navigation.h, Navigator,
    MissionManager...
  transceiver/transceiver.ino <- Station sol (pont USB <-> LoRa)
  IHM/                        <- Interface web de station sol (
    FastAPI)

```

La répartition suit une logique claire : *drivers* (matériel), *services* / *comm* (données), *navigation* (géométrie et mission), *control* (décisions) et *app* (orchestration). L'ordonnanceur est coopératif (sans RTOS) : `DroneApp::update()` déclenche chaque tâche quand sa période est échuée, évitant `delay()` qui bloquerait la lecture RF.

5.3. Décisions techniques fondamentales

5.3.1. MCPWM obligatoire — jamais LEDC

Listing 6: McpwmActuators.cpp – initialisation MCPWM

```

1 bool McpwmActuators::begin() {
2     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0A,
3         BoardConfig::SAIL_SERVO_PIN);
4     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0B,
5         BoardConfig::ROTOR_SERVO_PIN);
6     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM1A, BoardConfig::ESC1_PIN);
7     mcpwm_config_t cfg = {};
8     cfg.frequency      = Calibration::PWM_FREQ_HZ;    // 50 Hz
9     cfg.duty_mode      = MCPWM_DUTY_MODE_0;         // actif haut
10    cfg.counter_mode    = MCPWM_UP_COUNTER;
11    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_0, &cfg);   // voile + safran
12    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_1, &cfg);   // ESC
13    ...
14 }
```

Le périphérique MCPWM est indépendant des interruptions GPIO : les deux coexistent sans se perturber. Cette contrainte est **non négociable** dans ce projet.

5.3.2. Interruptions RC avec protection portMUX

Listing 7: RcReceiver.cpp – ISR et section critique

```

1 void IRAM_ATTR RcReceiver::handleEdge(uint8_t pin, Ch ch) {
2     const uint32_t now = micros();
3     const int      idx = static_cast<int>(ch);
4     if (gpio_get_level(static_cast<gpio_num_t>(pin))) {
5         portENTER_CRITICAL_ISR(&mux_);
6         riseUs_[idx] = now;           // front montant
7         portEXIT_CRITICAL_ISR(&mux_);
8     } else {
9         uint32_t rise;
10        portENTER_CRITICAL_ISR(&mux_);
11        rise = riseUs_[idx];
12        portEXIT_CRITICAL_ISR(&mux_);
13        const uint32_t width = now - rise;    // largeur
14        // d'impulsion
15        if (width >= RC_VALID_MIN_US && width <= RC_VALID_MAX_US) {
16            portENTER_CRITICAL_ISR(&mux_);
17            pulseUs_[idx] = static_cast<uint16_t>(width);
18            lastValidUs_[idx] = now;
19            portEXIT_CRITICAL_ISR(&mux_);
20        }
21    }
22 }
```

```

20     }
21 }

```

Les ISR sont placées en RAM (`IRAM_ATTR`) et se synchronisent avec la boucle principale via `portMUX` (mécanisme FreeRTOS approprié pour l'ESP32 dual-core). Un *timeout* de 100 ms marque un canal comme perdu.

5.3.3. Toutes les consignes d'actionneurs en microsecondes

Tous les actionneurs sont commandés exclusivement en microsecondes, jamais en degrés. Cela élimine une couche de conversion source d'erreurs et correspond directement au protocole PWM des servos et ESC.

5.3.4. Slew rate — protection contre les à-coups

Listing 8: `McpwmActuators.cpp` – limitation de variation

```

1  uint16_t McpwmActuators::slew(uint16_t current, uint16_t target,
   uint16_t step) {
2      if (current < target) {
3          uint32_t next = (uint32_t)current + step;
4          return (next >= target) ? target : (uint16_t)next;
5      }
6      if (current > target)
7          return (current > target + step) ? (uint16_t)(current -
           step) : target;
8      return current;
9  }
10 // ESC_SLEW_US = 30 us/tick, appelle toutes les 20 ms -> max 1500 us/s

```

6. Phase 1 — Contrôle manuel par radio

6.1. Réalisation

La Phase 1 couvre le contrôle manuel complet. On a implémenté les drivers (`AxpPower`, `RcReceiver`, `McpwmActuators`), le `ModeManager`, le `ManualController` et l'orchestrateur `DroneApp` (ticks de 20 ms pour le contrôle, 200 ms pour le debug, 1 s pour LoRa).

6.2. Modes de vol (sélecteur CH5) — version unifiée actuelle

Le sélecteur 3 positions est câblé sur CH5. La répartition des modes a été **réorganisée** par rapport à la version initiale (qui avait deux modes manuels séparés “ManualServo”

et “ManualProp”) :

CH5	Mode	Comportement
$\leq 1250 \mu\text{s}$	Automatic	Navigation autonome (suit la mission LoRa)
1400–1600	Sail	Inerte : toutes les sorties au neutre
$> 1800 \mu\text{s}$	Manual	Mode manuel unifié (voir ci-dessous)
Signal perdu	Failsafe	Suit la mission autonome (comme Automatic)

Attention sécurité : en cas de perte RC, le système passe en *Failsafe*, qui se comporte comme le mode autonome (navigue seul). Comme le mode autonome n’a jamais été testé sur l’eau, le **premier essai réel doit être manuel** et avec un plan de récupération.

6.3. Mode manuel unifié

Un seul mode (`computeManual`) contrôle à la fois la voile, le safran et le propulseur :

- **CH2 → voile (binaire)** : bande morte $\pm 35 \mu\text{s}$ autour de $1500 \mu\text{s}$. Le premier frame initialise l’état depuis la position du manche pour éviter un saut à l’entrée du mode. Dans la bande morte, le dernier état est maintenu.
- **CH4 → safran (positionnel)** : la course mesurée de CH4 (1180–1790 μs) est mappée sur la plage physique du treuil (`ROTOR_MIN/MAX_US` = 1417–1583 μs , $\pm 90^\circ$). Une bande morte centrale fixe le centre exact (1500 μs).
- **CH3 → propulseur (bidirectionnel, voir Section 6.4).**

Listing 9: ManualController – logique de voile binaire avec etat memorise

```

1 if (!sailStateInitialized_) {
2     sailState_ = (frame.ch2 >= RC_MID_US) ? +1 : -1;
3     sailStateInitialized_ = true;
4 }
5 if (frame.ch2 > RC_MID_US + db)      sailState_ = +1;
6 else if (frame.ch2 < RC_MID_US - db) sailState_ = -1;
7 cmd.sailUs = (sailState_ > 0) ? SAIL_PLUS_US : SAIL_MINUS_US;
```

6.4. Propulseur bidirectionnel (marche avant / arrière)

Dans la version initiale, le moteur n'allait que de 0 à 100 % (commande inversée, avec l'ESC en mode unidirectionnel, 1000 μ s = arrêt). On l'a modifié pour que le moteur aille de +100 % (**avant**) à -100 % (**arrière**) en utilisant l'ESC en mode **bidirectionnel**, où le neutre passe à 1500 μ s :

Position du manche CH3	Puissance	ESC
CH3_FULL_US (1100 μ s)	+100 % avant	2000 μ s
CH3_CENTER_US (1545 μ s) $\pm$ bande morte	0 % arrêt	1500 μ s
CH3_ZERO_US (1990 μ s)	-100 % arrière	1000 μ s

Comme le manche des gaz est de type *cran* (il ne se recentre pas), l'arrêt est placé au centre de sa course avec une bande morte (ESC_CENTER_DEADBAND_US = 40 μ s). Pour garder tout le firmware cohérent, le neutre 1500 μ s est aussi utilisé au démarrage, en Failsafe, dans le mode Sail et comme arrêt du mode autonome ; la limite inférieure de saturation a été abaissée à 1000 μ s pour permettre la marche arrière.

La marche arrière ne fonctionne physiquement que si l'ESC est **programmé en mode bidirectionnel** avec la carte EPRG-3. Sinon, toute valeur en dessous de 1500 μ s est ignorée et on n'obtient que la marche avant.

6.5. Résultats des tests — Phase 1

- [OK] L'aileron bascule correctement entre 1465 μ s et 1575 μ s.
- [OK] Le safran répond proportionnellement au manche CH4.
- [OK] Le Failsafe se déclenche en moins de 100 ms après la perte du signal.
- [OK] Compilation `arduino-cli` : $\approx$ 28 % flash, 7 % RAM.
- [] Calibration de banc de SAIL_PLUS/MINUS_US et de CH3_CENTER_US en attente.

7. Phase 2 — GPS et navigation

On a implémenté `GpsUart` (Serial1 sur GPI034/GPI012, TinyGPSPlus), `Navigator` (haversine : distance et cap), `MissionPlan` (jusqu'à 16 waypoints) et `MissionManager` (machine à états Idle→Running→Returning→Complete). Deux problèmes critiques de GPS ont été résolus dans cette phase.

7.1. Problème critique — GPS muet après configuration en flash

Symptôme : le compteur `chars` reste bloqué, le GPS est alimenté mais ne produit aucune phrase NMEA.

Cause : une session précédente (IDE Arduino ou u-center) a sauvegardé dans la flash interne du NEO-6M une configuration CFG-PRT qui désactive la sortie NMEA. Cette configuration survit aux coupures d'alimentation.

Envoyer un reset usine `UBX CFG-CFG` au démarrage (efface la configuration flash et charge les valeurs ROM). Implémenté dans `GpsUart::begin()`.

7.2. Problème critique — antenne active non alimentée

Symptôme : NMEA reçu mais `visible` = 0 satellites même en extérieur dégagé et avec antenne externe connectée.

Cause : le superviseur d'antenne du NEO-6M est désactivé par défaut. Sans l'activer, aucune tension de polarisation n'est fournie au LNA de l'antenne active et le commutateur RF du T-Beam reste sur l'antenne céramique interne, court-circuitant le connecteur u.FL.

Envoyer `UBX CFG-ANT` avec `flags` = 0x001B (contrôle de tension RF, détection de court-circuit, récupération). Après la correction : fix en moins de 60 s, 7 satellites, HDOP ≈ 2,2.

7.3. Note sur le démarrage à froid

Sans batterie LiPo connectée, chaque allumage perd l'almanach GPS : le démarrage à froid prend ≈ 2 min en extérieur. Avec la batterie, les éphémérides sont conservées et le fix s'obtient en quelques secondes (démarrage à chaud).

8. Phase 3 — LoRa et télémétrie

On a implémenté le driver `LoRaRadio` (SX1276, SPI HSPI, 433 MHz), la couche de protocole `LoRaComm` (heartbeat JSON à 1 Hz + dispatcher de commandes) et le sketch de station sol `transceiver.ino` (pont USB↔LoRa avec CLI compacte). Tout a été vérifié sur matériel (RSSI -101 à -106 dBm à courte distance).

8.1. Protocole JSON — heartbeat actuel

Le format a été affiné pour rester largement sous la limite de 255 octets de la FIFO du SX1276 (champs redondants supprimés, indicateurs de santé GPS/RC ajoutés, et le RSSI est injecté par le transcepteur au sol) :

Listing 10: Heartbeat drone -> sol (1 Hz)

```
{"origin":"boat","type":"info","message":{
  "mode":"standby|route-ready|navigate",
  "location":[48.36000,-4.56000],
  "servos":{"sail":10,"rudder":-12},
  "heading":270,"wind":180,"bat":7.42,
  "fix":1,"sat":7,"hdop":2.2,"rc":1,
  "wt":3,"wc":1,"wobs":42,"rssi":-104}}
```

- `servos.sail` = $\pm 10^\circ$ (binaire) ; `servos.rudder` en degrés du treuil. `fix/sat/hdop` = santé GPS ; `rc`=1 si le récepteur délivre des impulsions.
- `wt/wc` = waypoints total/actuel. `wobs` = progression de la mesure de vent (0–100 %), présent *seulement* pendant la mesure.
- `rssi` n'est **pas** envoyé par le bateau : il est injecté par le transcepteur au sol, il ne compte donc pas dans le budget de 255 octets.

8.2. Commandes (sol → drone)

Listing 11: Commandes supportées

```
{"origin":"server","type":"command","message":{"navigate":true}}
```

```

{"origin":"server","type":"command","message":{"stop":true}}
{"origin":"server","type":"command","message":{"restart":true}}
{"origin":"server","type":"command","message":{"wind-observation":
  true}}
{"origin":"server","type":"command","message":{"wind-command":{"value
  ":225}}}
{"origin":"server","type":"command","message":{"home":{"lat":48.36,"
  lon":-4.56}}}
{"origin":"server","type":"command","message":
  {"waypoints":{"number":2,"points":["48.36,-4.56,48.37,-4.55"]}}}

```

LoRa.endPacket() est synchrone. À SF7/BW125 kHz, un paquet de ≈ 210 octets prend ≈ 450 ms, bloquant la boucle principale une fois par seconde. Acceptable pour l'instant ; il faudra passer en TX asynchrone avant la navigation autonome sur l'eau si le jitter de la boucle de contrôle devient critique.

9. PCB Version 2 — Améliorations matérielles

La V2 du PCB corrige tous les problèmes de la V1 et intègre les enseignements du développement logiciel.

9.1. Correspondance des broches (PCB V2)

Signal	GPIO	Notes
RC CH2 (aileron)	39	SVN, entrée seule
RC CH3 (gaz)	14	Libéré (ne partage plus l'I2C)
RC CH4 (safran)	13	Libéré
RC CH5 (mode)	4	Commutateur 3 positions
Servo voile (PWM1)	2	Via transistor BC548
Servo safran (PWM2)	25	Via transistor BC548
ESC1	15	Via transistor BC548 (ESC unique)
ADC batterie	36	SVP, ADC1_CH0, entrée seule

Signal	GPIO	Notes
LoRa	5/19/27/18	SPI HSPI
SCK/MISO/- MOSI/CS		
LoRa RST	23	Maintenant libre (CH6 supprimé)
LoRa IRQ (DIO0)	26	
GPS RX / TX	34 / 12	Serial1
I2C SDA / SCL	21 / 22	AXP192 interne + connecteur d'extension

9.2. Amplification de niveau logique — transistors BC548

Les sorties PWM de l'ESP32 sont à 3,3 V ; les servos attendent 5 V. La V2 utilise des transistors NPN BC548 en émetteur commun.

Initialement, le canal du safran (OPR_B) utilisait `MCPWM_DUTY_MODE_1` (inversion), en supposant que le NPN inversait le signal et qu'il fallait pré-inverser. En réalité, le montage BC548 inverse déjà ; appliquer `DUTY_MODE_1` produisait une **double inversion** qui rendait le servo du safran inopérant. La solution est d'utiliser `DUTY_MODE_0` (actif haut) sur **les deux** canaux.

9.3. Mesure de batterie — diviseur haute impédance

On utilise un diviseur $R5 = 562 \text{ k}\Omega$ / $R6 = 120 \text{ k}\Omega$ (rapport 5,683) sur `GPIO36` avec une atténuation de 11 dB. À 7,4 V le diviseur consomme $\approx 10,9 \mu\text{A}$, négligeable.

Détail de montage : sur une carte, les résistances ont été soudées inversées (120 k en haut, 562 k en bas), donnant une lecture de 17,9 V au lieu de $\approx 5 \text{ V}$. Le code était correct ; la solution a été d'inverser physiquement les résistances.

9.4. AXP192 — I2C persistant

Dans la V1, on appelait `Wire.end()` après l'init de l'AXP192 pour libérer `GPIO21`/`GPIO22`. Dans la V2, ces broches restent libres pour l'I2C, donc `Wire.end()` est supprimé : le bus I2C reste actif et disponible pour de futurs capteurs sur le connecteur d'extension.

10. Navigation autonome (Phases 5 et 6)

10.1. Phase 5 — Estimation du vent à partir du track GPS

Sans capteur de vent, la direction est inferée en naviguant sur une amure fixe : le bateau fixe la voile à $+10^\circ$, lisse le cap GPS avec une moyenne mobile circulaire et, après avoir parcouru `WIND_OBS_DISTANCE_M` = 30 m, fixe le vent à `cap_lissé` $+90^\circ$. Il existe aussi une commande LoRa `wind-command` pour forcer la direction manuellement (secours fiable).

Le décalage de $+90^\circ$ et le seuil de 30 m nécessitent une validation sur le terrain. De plus, pendant la mesure, le heartbeat est identique au repos sauf pour le champ `wobs`.

10.2. Phase 6 — Algorithme de navigation à voile

L'algorithme de navigation (`navigation.h`) a été développé par un membre du groupe (branche `Testautoboat` du dépôt `DeltaGod/PER_MEA_LYINCROYAP`) et intégré à ce firmware. Il implémente :

- Zones interdites au près et au portant (*upwind/downwind*) avec zigzag de louvoyage.
- Évitement d'empannage (*empannage*) via une machine à états.
- Couloir de tolérance latérale (*cross-track*) autour de la ligne vers le waypoint.
- Lofer/abattre (*lofer/abattre*) pour le suivi fin du cap.

La dernière version intégrée apporte : une sélection d'amure initiale plus intelligente (`nav_sideMovingTowardWaypoint`, qui choisit le côté qui progresse le plus vers le waypoint), une détection d'arrivée par "plan du waypoint dépassé à l'intérieur du couloir" (`nav_hasPassedWaypoint`) et un couloir par défaut élargi de 20 à 100 m. L'enveloppe `AutoController` mappe le cap de safran de l'algorithme 1:1 sur les degrés physiques du treuil, limité à $\pm 20^\circ$ (`ROTOR_AUTO_RANGE_DEG`), car le $\pm 90^\circ$ complet se révélait trop agressif.

La navigation autonome est **codée mais jamais testée sur l'eau** et n'a pas de tests unitaires. Il manque aussi un *WinchTracker* : le Regatta ECO II n'a pas de retour de position, donc la position réelle du safran n'est pas connue (voir Section 13).

11. Interface de station sol (IHM)

La station sol se compose d'un second T-Beam avec `transceiver.ino` (pont USB↔LoRa) et d'une application web (IHM/) en Python FastAPI + MongoDB + une carte Leaflet. Fonctions principales :

- Carte avec la position du bateau, waypoints (clic pour les définir), envoi de route et *polling* de télémétrie à 1 Hz.
- Panneau de santé avec voyants (Liaison, GPS, Batterie, RC, Contrôle) et RSSI.
- Boussole de vent et prédiction de trajectoire (réplique en JavaScript de `navigation.h`).
- Barre de progression de la mesure de vent.
- Autodétection du port du transcepteur par numéro de série du CP2104 (transcepteur = 01C00B54, bateau = 02126CF1).

12. Problèmes résolus et solutions détaillées

Cette section réunit, de manière exhaustive, les problèmes significatifs rencontrés pendant le développement et comment ils ont été résolus.

12.1. Alimentation du T-Beam — impossible d'alimenter par les broches

Problème : le T-Beam V1.1 **ne peut pas être alimenté en injectant 5 V par les broches du header d'extension**. Les seules voies d'alimentation valides sont le **porte-batteries 18650/LiPo** (connecteur JST de la carte elle-même) et le **port USB**. Le PCB V2, en revanche, a été conçu pour alimenter le T-Beam via

la broche **+5V** du header d'extension (sortie du régulateur abaisseur Pololu). La raison est la topologie interne d'énergie du T-Beam : la broche **+5V** du header se trouve sur le chemin du VBUS de l'USB, après la diode de protection, et **n'entre pas de façon fiable dans l'AXP192** (le PMIC qui active les rails de 3,3 V, GPS et LoRa). Par conséquent, alimenté seulement par cette broche, l'AXP192 **ne démarre souvent pas**, et sous charge la tension s'effondre (on a observé le bateau tournant à $\approx 0,81$ V, clairement en *brownout*), ce qui provoque en plus la ré-énumération continue du convertisseur USB-série (messages noyau `cp210x ... status -19`).

Solution rapide adoptée : on a coupé un câble USB-A $\rightarrow$ USB-C ; le bout USB-A (lignes **+5V** et **GND**) a été **soudé directement au rail de 5 V du PCB** (sortie du Pololu), et le bout USB-C se connecte au port USB du T-Beam. Ainsi, l'énergie du PCB entre dans le T-Beam par son **chemin d'alimentation USB valide** (VBUS $\rightarrow$ AXP192), au lieu de la broche **+5V** du header, qui n'arrivait pas à démarrer le PMIC.

Recommandation : cette solution est un *patch* fonctionnel mais peu élégant (un câble USB soudé au rail). Une correction définitive dans une future révision du PCB serait de router l'alimentation du PCB directement vers l'entrée correcte du T-Beam (VBUS ou le JST de batterie), en évitant la broche **+5V** du header.

12.2. Moteur qui fonctionne “par impulsions” — coupure par basse tension

Symptôme : avec la batterie à ≈ 7 V (selon la télémétrie), le moteur ne tourne pas de façon continue : il démarre un instant, se coupe, et ne repart que si l'on ramène la commande à 0 % puis de nouveau à 100 % un peu plus tard.

Cause (ce n'est pas un bug logiciel) : c'est la **coupure par basse tension (LVC)** de l'ESC. Une batterie 2S à 7 V est déjà basse ($\approx 3,5$ V/cellule) ; sous la charge du moteur, la tension passe sous le seuil de coupure de l'ESC ($\approx 3,0$ – $3,2$ V/cellule) et l'ESC coupe le moteur pour protéger le pack. Sans charge, la tension se rétablit, la coupure se réarme et le moteur retourne tourner un instant, produisant justement ce comportement “par impulsions”. L'effet empire beaucoup si l'ESC a un mauvais réglage du nombre de cellules.

Solution : chaîner en série 2 batteries LiPo ($\approx 15,4$ V). Aucun changement de code ne corrige une coupure par basse tension ; le firmware envoie une consigne stable.

12.3. Échec de programmation du bateau — brownout pendant le flash

Symptôme : la programmation échoue répétitivement (“*chip stopped responding*”, “*No serial data received*”) et le CP2104 se ré-énumère au milieu de l’écriture. Cela n’arrive que lorsque le T-Beam est connecté directement au PCB.

Cause : le même *brownout* d’alimentation. Le pic de courant de l’effacement/écriture de la flash effondre la tension et l’ESP32 se réinitialise.

Déconnecter le T-Beam du PCB, le programmer et, sans le déconnecter de l’USB, le replacer sur le PCB.

12.4. Autres problèmes de firmware déjà résolus

12.4.1. DUTY_MODE_1 sur OPR_B — servo du safran inopérant

Double inversion à travers le BC548 (voir Section 9.2). Solution : DUTY_MODE_0 sur les deux canaux.

12.4.2. Seuil d’armement de l’ESC trop bas

Le minimum réel du manche des gaz du PTR-6A est $\approx 1265 \mu\text{s}$, donc le seuil d’armement de $1050 \mu\text{s}$ n’était jamais atteint. Il a été porté à $1300 \mu\text{s}$. **Note :** le geste d’armement a été **supprimé** ensuite, au passage au mode manuel unifié et au contrôle bidirectionnel (il était incompatible avec le nouveau mapping des gaz).

12.4.3. Mode bloqué en Automatic — confusion CH4/CH5

La version initiale lisait CH4 pour le mode, alors que le commutateur est sur CH5. Solution : lire `frame.ch5`.

12.4.4. Port série occupé lors de la programmation

Un moniteur série ou un script Python gardait le port ouvert (Errno 16: Device or resource busy). Solution : fermer le moniteur ou fuser `/dev/ttyUSB0` et tuer

le processus.

13. Problèmes rencontrés — options d'amélioration

Cette section réunit les problèmes encore **ouverts** et, pour chacun, l'option d'amélioration proposée pour le résoudre. Ce sont les axes de travail prioritaires pour que le drone puisse réellement accomplir sa mission.

13.1. Portée LoRa très faible à l'intérieur du boîtier électronique

Problème : avec l'antenne LoRa actuelle placée à l'intérieur du boîtier électronique, la portée radio est très faible. Le boîtier fait écran et la proximité de l'électronique dégrade le diagramme de rayonnement, réduisant drastiquement le RSSI et la portée utile de télémétrie et de commandes.

Amélioration proposée : acquérir une **antenne LoRa externe** (433 MHz) et la placer **sur le mât principal**, le plus haut et dégagé possible, hors du boîtier. Cela améliore la ligne de vue et éloigne l'antenne des masses métalliques et du bruit de l'électronique.

À considérer — connecteur : il faut vérifier le connecteur de l'antenne et du module. Le SX1276 du T-Beam expose généralement **u.FL/IPEX**, tandis que les antennes externes 433 MHz ont habituellement du **SMA**. Il faudra le **câble adaptateur / pigtail correspondant (u.FL → SMA)** et veiller à ce qu'il soit en 433 MHz (pas en 2,4 GHz) pour ne pas dégrader l'adaptation.

13.2. Incohérences du GPS

Problème : le GPS présente des incohérences (fix intermittent, peu de satellites, position peu stable), aggravées par l'emplacement de l'antenne dans ou près du boîtier et de l'électronique.

Amélioration proposée : acquérir **une autre antenne externe, cette fois de GPS**, et la placer **sur le pont**, avec une vue dégagée du ciel et loin de l'antenne LoRa et de l'électronique.

À considérer — connecteur : comme pour LoRa, il faut vérifier le connecteur. L'entrée d'antenne GPS du NEO-6M est en **u.FL** ; les antennes GPS actives viennent souvent en **SMA** ou directement avec un câble u.FL. Il faudra prévoir l'**adaptateur adéquat** et confirmer que l'antenne est **active** (le superviseur d'antenne est déjà activé par logiciel, voir Section 7.2).

13.3. Sans mesure du vent ni de la position de la voile

Problème : le schéma actuel **ne mesure pas d'où vient le vent ni la position réelle de la voile**. Comme le treuil du safran (Regatta ECO II) n'a pas non plus de retour de position, **il est impossible de connaître la position réelle du safran** ni de fermer la boucle de contrôle avec des données réelles : toute la navigation autonome dépend d'estimations (vent déduit du track GPS, position de safran supposée à partir de la consigne).

Amélioration proposée : se procurer **2 capteurs** qui mesurent respectivement la **direction du vent** et la **position de la voile**, et les connecter au **bus I2C de la carte** via le **connecteur déjà prévu sur le PCB** (GPIO21/GPIO22, que le firmware a laissés libres en supprimant `Wire.end()`). Avec la direction réelle du vent et la position réelle de la voile, il serait alors possible de déduire la position effective du safran et de fermer correctement la boucle de navigation.

Avantage architectural : le firmware réserve déjà le bus I2C et le PCB expose déjà le connecteur d'extension, donc l'intégration électrique est directe ; il manquerait le driver **SensorBus** côté logiciel.

13.4. Peu d'espace pour les batteries dans le boîtier

Problème : le boîtier électronique a **peu d'espace pour les batteries**, ce qui limite la capacité embarquée et donc l'autonomie et la marge de tension sous charge (en lien direct avec la coupure par basse tension de la Section 12.2).

Amélioration proposée : revoir la conception mécanique du boîtier pour loger un pack de plus grande capacité (ou une meilleure répartition des cellules), en tenant compte en plus de la place qu'occuperont les nouveaux capteurs et les connecteurs des antennes externes. Il convient de dimensionner le pack pour maintenir la tension au-dessus du seuil LVC de l'ESC durant toute la mission.

14. État actuel du système

Phase	Objectif	État	Notes
1	Contrôle manuel RC	[OK] Complet	Mode unifié + moteur bidirectionnel
2	GPS + navigation	[OK] Complet	Fix testé en extérieur
3	LoRa + télémétrie	[OK] Complet	TX/RX vérifiés
PCB V2	Nouveau câblage + I2C libre	[OK] Complet	Compilé et testé
5	Estimation du vent	[OK] Codé	Non validé sur l'eau
6	Navigation autonome	[OK] Codé	Non testé sur l'eau, sans tests
4	Capteurs I2C	[] En attente	Voir Section 13
7	Journalisation / mission complète	[] En attente	SPIFFS, propulsion auto

14.1. Valeurs de calibration actuelles (résumé)

Paramètre	Valeur	Notes
SAIL_CENTER_US	1520 μ s	Centre Futaba (pas 1500)
SAIL_PLUS/MINUS_US	1575 / 1465	$\pm 10^\circ$ — calibrer au banc
ROTOR_MIN/MAX_US	1417 / 1583	$\pm 90^\circ$ physique (manuel)
ROTOR_AUTO_RANGE_DEG	20°	Course utilisée en auto

Paramètre	Valeur	Notes
ESC_REVERSE_MIN_US	1000 μs	−100 % (arrière, bidirectionnel)
ESC_NEUTRAL_US	1500 μs	Neutre / arrêt
ESC_MAX_US	2000 μs	+100 % (avant)
CH3_CENTER_US	1545 μs	Centre des gaz — vérifier au banc
ESC_SLEW_US	30 $\mu\text{s}/\text{tick}$	Limitation de variation
Rapport diviseur batterie	5,683	(562k + 120k) / 120k